Universidad San Francisco Xavier de Chuquisaca · COM450 – Calidad de Software

Práctica 9 — Inteligencia Artificial aplicada a la Calidad de Software

Pruebas y manuales de usuario generados con IA, medidos y verificados

Estudiante: David Luján (trabajo individual) **Fecha:** 16 de octubre de 2026
Repositorio: https://github.com/lujan-99/descuentos-app — rama `practica9`, pull request #6 **Aplicación:** https://descuentos-web-blush.vercel.app (API: https://descuentos-api-il43.onrender.com)

Herramientas elegidas: pruebas → Claude (modelo Opus 5.5, usado desde Claude Code); manuales → Claude (asistente de lenguaje con inventario y capturas).

> **Declaración de uso de IA.** Esta práctica se realizó con asistencia de Claude Code (modelo Claude Opus 5.5) bajo mi supervisión: además de generar las pruebas y el manual (lo que pide la práctica), el asistente ejecutó los comandos, sacó las capturas, escribió los scripts de medición y redactó el borrador de este informe. Las decisiones de calidad (qué es correcto, qué se conserva, cómo se corrige el defecto de zona horaria, qué umbral exige el CI) están justificadas en cada sección y las revisé yo. La bitácora del Anexo 1 registra cada interacción relevante.

Resumen de resultados

Indicador de "artefacto sano"	Resultado	Dónde
Pruebas ejecutadas, 0 omitidas	58 pruebas de front-end (19 → 58), 0 omitidas; reto: 6 pruebas, 0 omitidas	§3, §8
0 fallos y `tsc` limpio	Sí, en local y en el CI (PR #6)	§5
Cobertura ≥ 80 % exigida	98,24 % líneas (umbral 80 % configurado en Vitest); `cupones.ts` 100 %	§3
Mutación ≥ 80 %	`cupones.ts`: 100 % (UTC) / 98,31 % (La Paz, 1 equivalente justificado); reto: 60,71 % → 82,14 %	§3, §8
Manual — veracidad	Versión IA 1: 32 de 43 afirmaciones correctas (74,4 %); versión final: 100 % verificada	§6
Hallazgo principal	Las pruebas de la ronda 1 **fijaron un defecto real** (vencimiento en UTC) como comportamiento esperado; las de la ronda 2 lo encontraron; el manual y el inventario repitieron un dato falso (decimales con coma) que solo se descubrió verificando	§3, §6

1. Investigación y elección de herramientas (Paso 1)

1.1 Respuestas de la investigación

Herramienta	Tipo de IA y entrada	Compatibilidad / salida	Plan gratuito y límites	Intervención humana	Datos que salen	Limitaciones
Claude (Opus 5.5) en Claude Code	LLM conversacional/agente; recibe código, reglas, archivos e imágenes	Cualquier lenguaje; generó TS + Vitest y Cypress sin errores de compilación	Plan gratuito en claude.ai con límite diario; Claude Code requiere plan Pro/Max	Revisar y medir todo; da explicaciones de cada cambio	Prompts y archivos a Anthropic; en planes de consumo se pueden usar para entrenar salvo que se desactive "Help improve Claude"; retención ~30 días sin entrenamiento	Si lee el código, reproduce sus defectos (ronda 1)

Herramienta	Tipo de IA y entrada	Compatibilidad / salida	Plan gratuito y límites	Intervención humana	Datos que salen	Limitaciones
OpenCode + Big Pickle	Agente de terminal con modelo gratuito de OpenCode Zen	TS + Vitest correcto	Gratis (modelo "free" temporal)	En modo *build* instaló `vitest` y ejecutó pruebas **sin pedir permiso**	Zen: durante el periodo gratuito los datos pueden usarse para mejorar el modelo	Menos control del contexto: actúa sobre la carpeta
OpenCode + MiMo v2.6 Flash	Mismo agente, otro modelo gratuito	Pruebas correctas pero usó `process` (Node) → **rompe `tsc`**	Gratis	Hay que corregir la compilación	Igual que arriba (modelo gratuito)	Supone un entorno Node que el proyecto no tipa
OpenCode + DeepSeek V4.1 Flash	Mismo agente, otro modelo gratuito	TS + Vitest; **1 de 18 pruebas falla** (espera `Bs 1,01` para 1,005)	Gratis	Hay que corregir el valor esperado antes de medir	Igual que arriba	Inventa una regla de redondeo para `formatearBs`
GitHub Copilot (no probado)	Asistente de IDE	TS/Vitest	Free: ~2000 completados y ~50 chats al mes (fuentes secundarias; cambió a créditos en 2026)	—	Código del editor a GitHub	No se probó: sin cuenta configurada
Nemotron 3 Ultra y Step 5 (OpenCode)	Modelos gratuitos	—	—	—	—	Respondieron `[400] Provider returned error`: no utilizables ese día

Categoría B (manuales):

Herramienta	Entrada	Salida	Plan gratuito	Datos	Limitaciones
Claude	Inventario + capturas	Markdown (luego HTML/PDF)	Sí (con límite)	Texto e imágenes a Anthropic	Solo tan veraz como el inventario
OpenCode + DeepSeek / MiMo	Inventario + capturas adjuntas	Markdown	Gratis	Modelos "free" pueden entrenar con los datos	DeepSeek antepone texto de sistema ("Estoy en modo Plan…")
Scribe (no probado)	Grabación de pasos en el navegador	Guía con capturas; PDF/HTML/Markdown **solo en Pro**	Free: guías ilimitadas en el navegador, sin exportación ni redacción de datos	Capturas en la nube	Sin exportación gratis; no conoce reglas de negocio
Tango (no probado)	Grabación de pasos	Guía web	Free: ~5 flujos en la biblioteca del equipo; exportación discutida	Capturas en la nube	Límite de flujos

Herramienta	Entrada	Salida	Plan gratuito	Datos	Limitaciones
Guidde (no probado)	Grabación + voz	Video guía	Free: 25 videos, **marca de agua, sin descarga**	Video en la nube	No sirve para un PDF

1.2 Matriz ponderada — categoría A (pruebas)

Pesos definidos antes de puntuar. Puntaje 0–5; total ponderado = Σ(peso × puntaje) / 5.

Criterio	Peso	Claude	Big Pickle	MiMo	DeepSeek
Compatibilidad con TS / Vitest (compila, corre)	25	5	5	3	4
Control sobre el contexto (reglas, archivos, no actuar solo)	20	5	3	4	4
Costo y límites del plan gratuito	15	3	5	5	5
Privacidad (qué sale, entrenamiento, opción de desactivar)	15	3	1	2	2
Calidad en la muestra propia (`descuentos.ts`)	25	5	5	3	2
Total ponderado	100	**88**	**80**	**67**	**67**

Muestra: el mismo prompt mínimo ("Genera pruebas unitarias con Vitest para este archivo TypeScript (descuentos.ts)") en una conversación nueva por herramienta, medido con el mismo script (`medir-muestra.sh`: Vitest, `tsc`, Stryker solo con ese archivo):

Herramienta	Pruebas	Verdes	`tsc`	Mutación `descuentos.ts`	Aserciones débiles
Claude	26	26	limpio	100 % (28/28)	0
Big Pickle	37	37	limpio	100 % (28/28)	0
MiMo	26	26	**6 errores TS2591**	100 % (28/28)	0
DeepSeek	18	**17**	limpio	no medible (Stryker exige suite verde)	0

Justificación. Claude y Big Pickle empatan en calidad medida. Lo decisivo fue **privacidad y control**: Big Pickle es gratuito porque los datos pueden usarse para entrenar el modelo, y su agente instaló dependencias y ejecutó comandos en la carpeta sin pedir permiso; en un proyecto de cliente eso es inaceptable sin una política (Ejercicio 5). Claude acepta reglas de negocio, capturas y archivos, explica cada cambio y permite desactivar el entrenamiento. El costo juega en contra (Claude Code requiere plan de pago) y lo declaro: ya tengo esa suscripción, así que la comodidad también pesó. MiMo y DeepSeek quedan descartados por un defecto observable en la muestra (rompe el build / afirma un valor falso).

1.3 Matriz ponderada — categoría B (manuales)

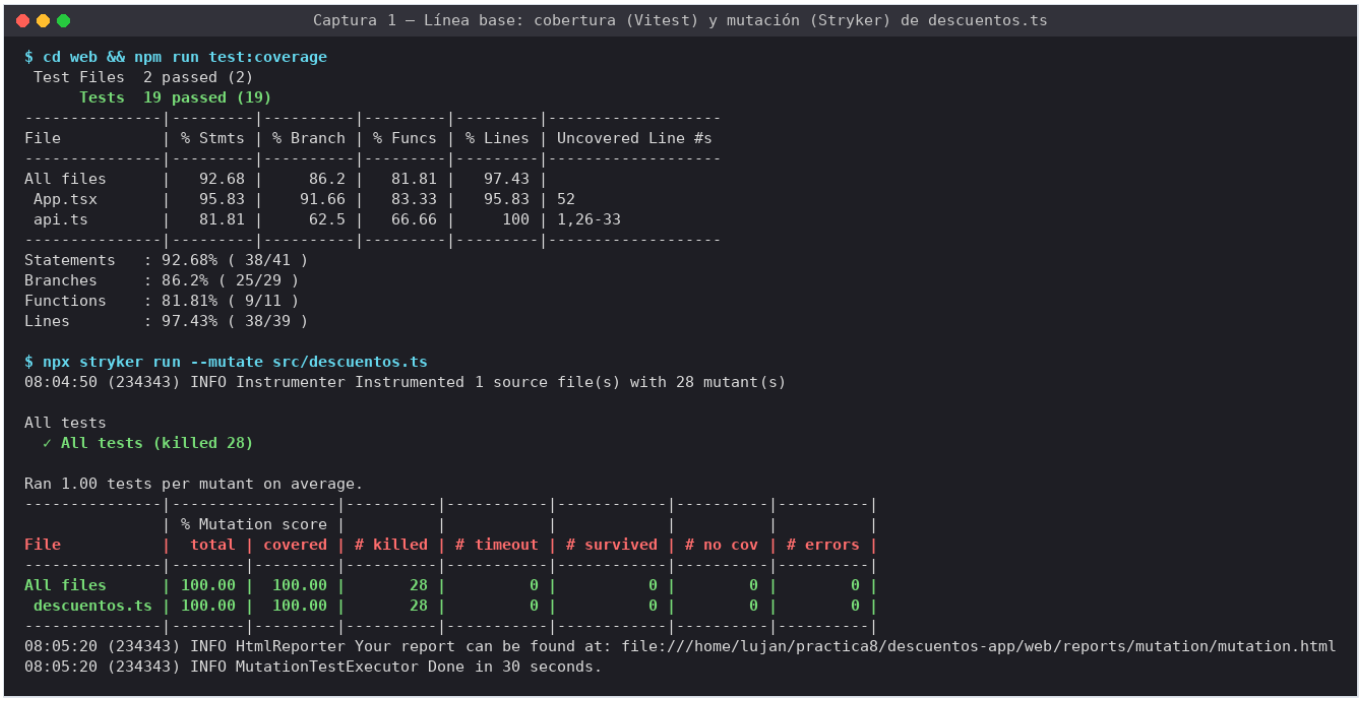
Criterio	Peso	Claude	DeepSeek	MiMo	Scribe	Tango	Guidde
Formato de salida útil (PDF/HTML editable)	20	5	5	5	2	2	1
Control del contexto (acepta inventario y reglas)	20	5	4	4	2	2	2
Costo / exportación en plan gratuito	20	4	5	5	1	2	1
Privacidad	15	3	2	2	2	2	2
Calidad en la muestra propia	25	4	3	4	n/p	n/p	n/p
Total ponderado	100	**84**	**76**	**80**	35/75	38/75	27/75

n/p: no probada (requiere cuenta y extensión); su total se da sobre 75 y, aun así, el plan gratuito sin exportación la descarta para un manual en PDF. Calidad: los tres asistentes respetaron el inventario y no inventaron funciones, pero todos repitieron el dato falso del inventario sobre la coma decimal (§6);

DeepSeek lo escribió como instrucción ("puedes escribir números con decimales separados por coma") y antepuso texto de sistema; MiMo marcó la duda como "NO VERIFICADO". **Elegida: Claude**, por el control del contexto (pide capturas, marca lo no verificado) y porque es la misma herramienta de la categoría A (una sola política de datos).

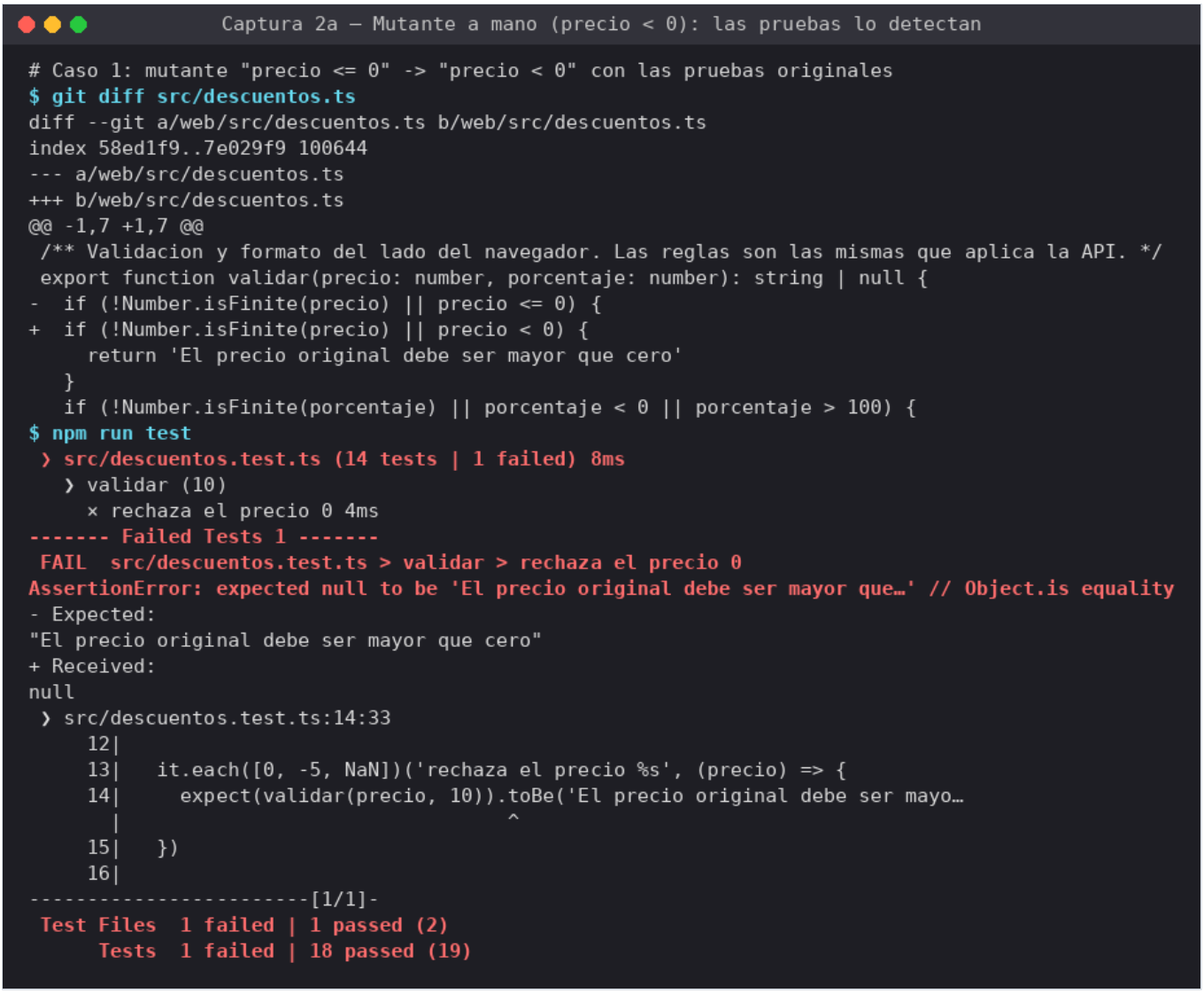
2. Línea base y mutante a mano (Pasos 2 y 3)

Rama `practica9`, `@stryker-mutator/core` 10.0.0, `stryker.config.json` y `cupones.ts` del material. Exclusiones de `.stryker-tmp` y `reports` en `.gitignore`, `.prettierignore` y `eslint.config.js` (commit `a7b1cfc`).



Captura 1 — 19 pruebas en verde, 97,43 % de líneas; Stryker: 28 mutantes de `descuentos.ts`, 100 %.

Hallazgo de la línea base: al copiar `cupones.ts` **sin pruebas**, `npm run test:coverage` falla (67,85 % < 80 %). El umbral exigido (no solo medido) impide que un módulo nuevo entre sin pruebas.


Captura 2a — Mutante a mano (precio < 0): las pruebas lo detectan
Caso 1: mutante "precio <= 0" -> "precio < 0" con las pruebas originales
$ git diff src/descuentos.ts
diff --git a/web/src/descuentos.ts b/web/src/descuentos.ts
index 58ed1f9..7e029f9 100644
--- a/web/src/descuentos.ts
+++ b/web/src/descuentos.ts
@@ -1,7 +1,7 @@
 /** Validacion y formato del lado del navegador. Las reglas son las mismas que aplica la API. */
 export function validar(precio: number, porcentaje: number): string | null {
- if (!Number.isFinite(precio) || precio <= 0) {
+ if (!Number.isFinite(precio) || precio < 0) {
     return 'El precio original debe ser mayor que cero'
   }
   if (!Number.isFinite(porcentaje) || porcentaje < 0 || porcentaje > 100) {
$ npm run test
 ❯ src/descuentos.test.ts (14 tests | 1 failed) 8ms
   ❯ validar (10)
     × rechaza el precio 0 4ms
------- Failed Tests 1 -------
 FAIL src/descuentos.test.ts > validar > rechaza el precio 0
AssertionError: expected null to be 'El precio original debe ser mayor que…' // Object.is equality
- Expected:
"El precio original debe ser mayor que cero"
+ Received:
null
 ❯ src/descuentos.test.ts:14:33
     12|
     13| it.each([0, -5, NaN])('rechaza el precio %s', (precio) => {
     14| expect(validar(precio, 10)).toBe('El precio original debe ser mayo…
       | ^
     15| })
     16|
----------------------[1/1]-
 Test Files 1 failed | 1 passed (2)
      Tests 1 failed | 18 passed (19)

Captura 2b — Pruebas debilitadas (sin el 0): el mutante sobrevive

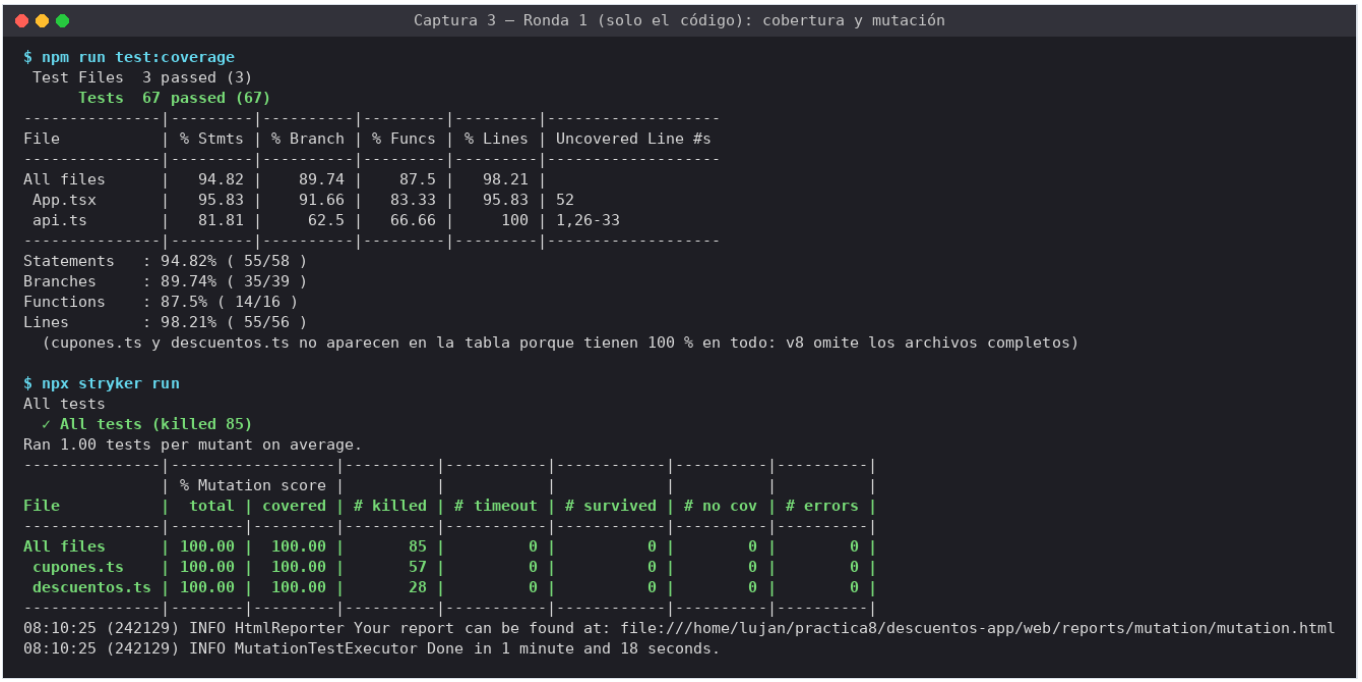
```
# Caso 2: mismo mutante con las pruebas debilitadas (sin el 0)
$ git diff src/
diff --git a/web/src/descuentos.test.ts b/web/src/descuentos.test.ts
index 2aa1c47..0227722 100644
--- a/web/src/descuentos.test.ts
+++ b/web/src/descuentos.test.ts
@@ -10,7 +10,7 @@ describe('validar', () => {
     expect(validar(50, porcentaje)).toBeNull()
   })
-  it.each([0, -5, NaN])('rechaza el precio %s', (precio) => {
+  it.each([-5, NaN])('rechaza el precio %s', (precio) => {
     expect(validar(precio, 10)).toBe('El precio original debe ser mayor que cero')
   })
diff --git a/web/src/descuentos.ts b/web/src/descuentos.ts
index 58ed1f9..7e029f9 100644
--- a/web/src/descuentos.ts
+++ b/web/src/descuentos.ts
@@ -1,7 +1,7 @@
 /** Validacion y formato del lado del navegador. Las reglas son las mismas que aplica la API. */
 export function validar(precio: number, porcentaje: number): string | null {
-  if (!Number.isFinite(precio) || precio <= 0) {
+  if (!Number.isFinite(precio) || precio < 0) {
     return 'El precio original debe ser mayor que cero'
   }
   if (!Number.isFinite(porcentaje) || porcentaje < 0 || porcentaje > 100) {
$ npm run test
 Test Files  2 passed (2)
      Tests  18 passed (18)
```

Captura 2 — `precio <= 0` → `precio < 0`: con las pruebas originales falla "rechaza el precio 0" (detectado); sin el 0 en `it.each`, las 18 pruebas pasan (sobrevive).

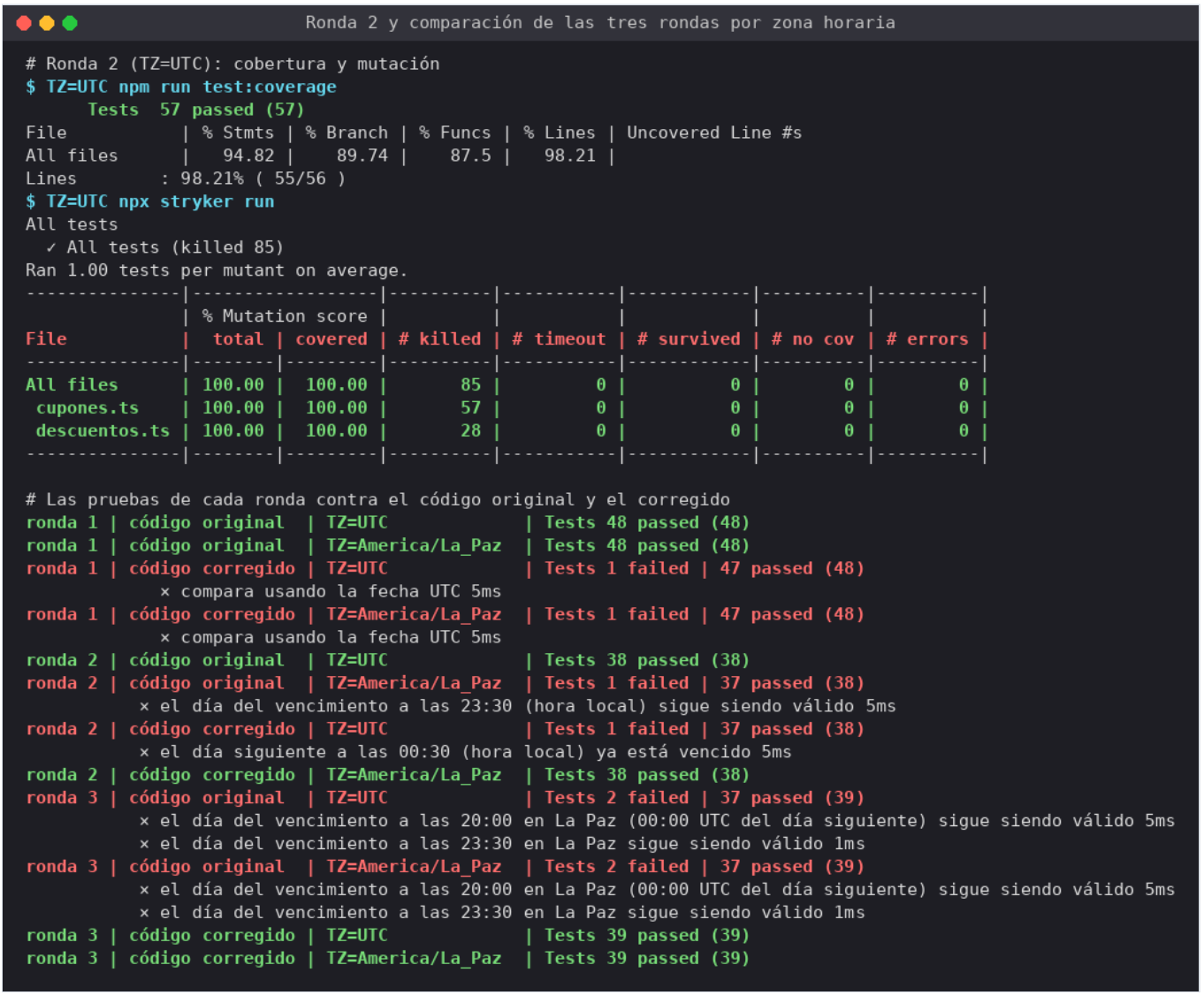
3. Tres rondas de generación de pruebas (Pasos 4 a 6)

Cada ronda en una conversación nueva con Claude Opus 5.5 (prompts exactos en `practica9/rondas/` y en la bitácora).

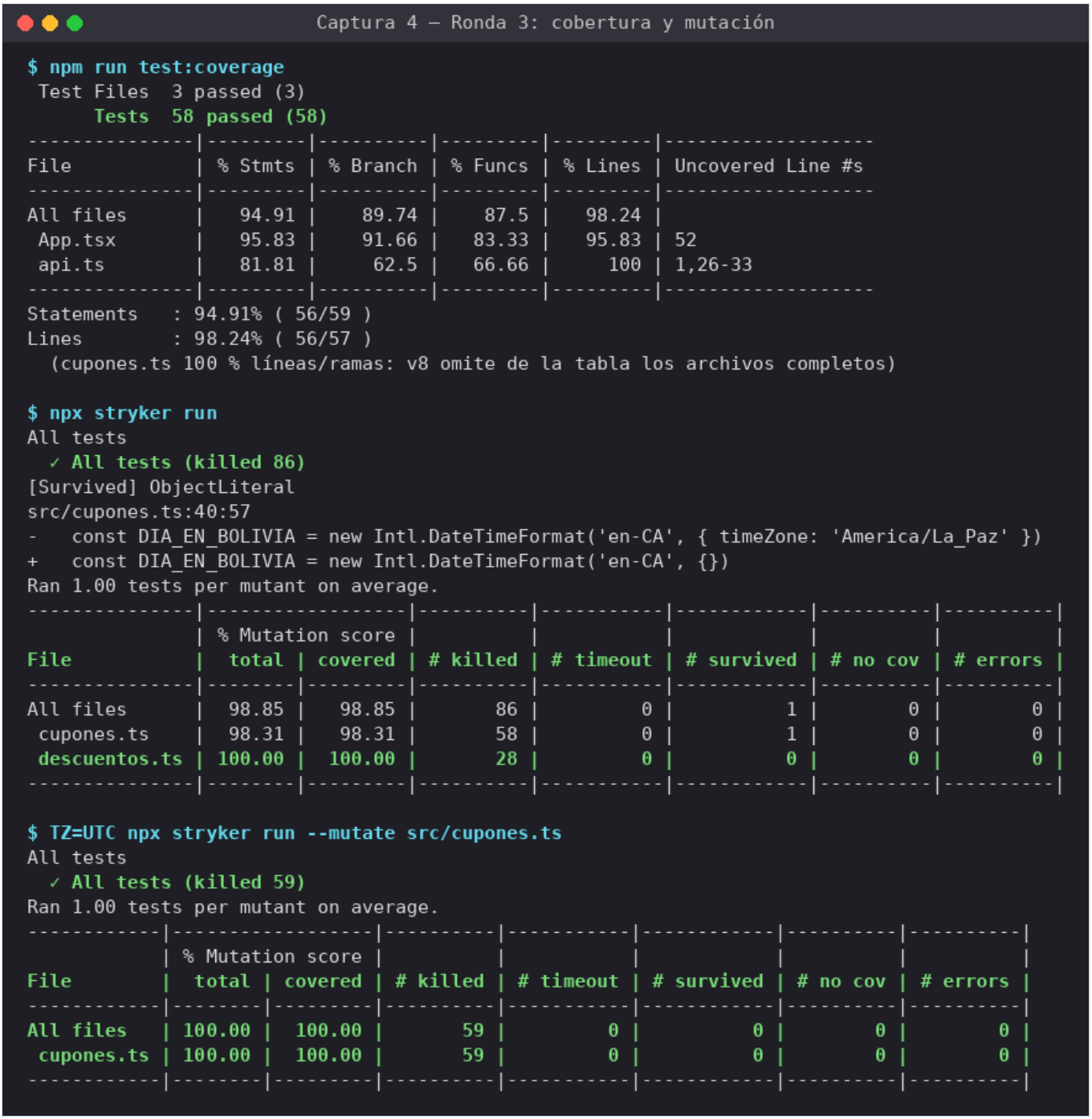
Ronda	Qué se le dio	Pruebas	Verdes	Cobertura `cupones.ts` (líneas)	Mutación `cupones.ts`	Sobrevivientes
1	Solo el código (sin la cabecera de reglas)	48	48	100 %	100 % (57/57)	0
2	+ reglas de negocio y límites	38	38 en UTC; **37 en La Paz**	100 %	100 % (57/57, TZ=UTC)	0
3	+ evidencia (0 sobrevivientes y la prueba que falla por zona horaria)	39	39 en cualquier zona (con el código corregido)	100 %	98,31 % local (58/59) · **100 % en UTC (59/59)**	1 (equivalente solo en La Paz)



Captura 3 — Ronda 1: 67 pruebas en verde (48 nuevas), `cupones.ts` con 100 % de líneas y 100 % de mutación.



Las pruebas de cada ronda contra el código original y el corregido, en UTC y en America/La_Paz.



Captura 4 — Ronda 3: 58 pruebas, mutación 98,31 % en La Paz (1 sobreviviente) y 100 % con `TZ=UTC`.

Conclusión de las tres rondas. El resultado contradice lo "habitual" que anticipa la guía: con un modelo fuerte, la ronda 1 ya alcanzó 100 % de cobertura **y** 100 % de mutación. Pero el puntaje no dijo nada sobre si los valores esperados eran correctos, que es exactamente el límite de la mutación ("una prueba que fija un valor equivocado también detecta mutantes"):

- **Ronda 1 (solo código) fijó un defecto.** La prueba "compara usando la fecha UTC" afirma que un cupón que vence el 16/10 ya no vale a las 00:00 UTC del 17, es decir, a las **20:00 del 16 en Bolivia**. La regla 3 dice "vale hasta el día de su vencimiento inclusive". La IA leyó `toISOString()` y convirtió el comportamiento del código en el valor esperado: sesgo de confirmación en estado puro. Con el código corregido, esa prueba falla.
- **Ronda 2 (con reglas) encontró el defecto**, pero sus fechas usaban la hora local de la máquina: falla en La Paz y pasa en el CI (UTC). Una prueba que depende del entorno no es determinista (lista de control, punto 8).

- **Ronda 3 (con la evidencia)** fijó las fechas con desfase `-04:00`: falla con el código original en cualquier zona y pasa con el corregido en cualquier zona. Decisión humana: la regla 3 se precisó como "calendario de Bolivia" y se corrigió `fechaIso` con `Intl.DateTimeFormat('en-CA', { timeZone: 'America/La_Paz' })` (commit `3536cf1`).
- **El sobreviviente de la ronda 3** quita `{ timeZone: 'America/La_Paz' }`. En una máquina configurada en La Paz el resultado es idéntico (mutante equivalente *en ese entorno*); en UTC se detecta. Por eso el job de mutación del CI fija `TZ: UTC` (Ejercicio 1). Se justifica, no se ignora.
- Además, en la ronda 3 la IA justificó la zona horaria citando "el comentario original"; ese comentario lo había escrito ella misma en la ronda 2, no estaba en las reglas. Se registró como afirmación sin fuente.

4. Revisión de las pruebas con la lista de control (Anexo B)

Revisión de las pruebas de la **ronda 1** (la que solo vio el código). `it.each` cuenta como una fila.

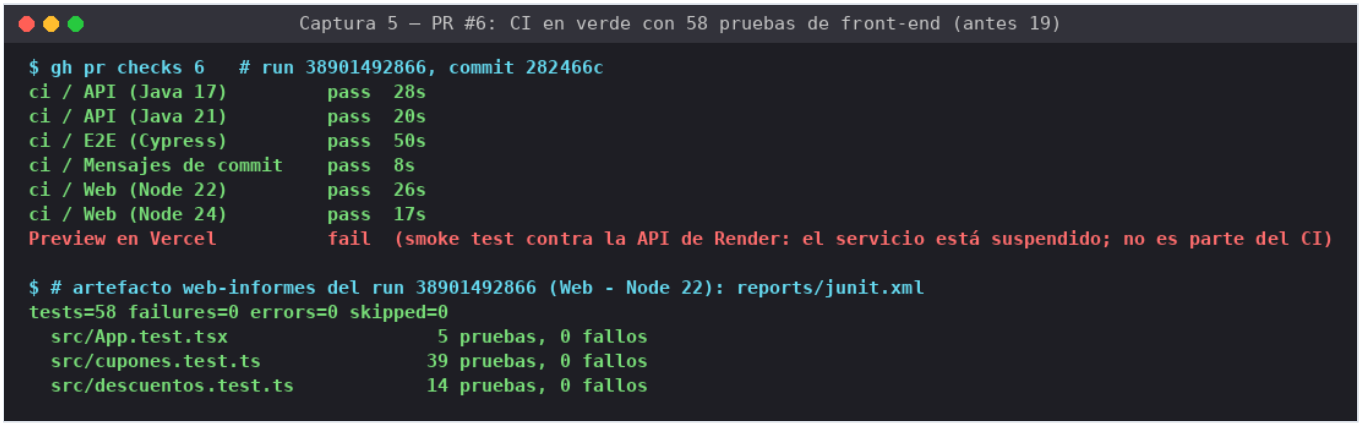
Prueba (ronda 1)	¿Aserción específica?	¿Valor de la regla?	¿Límite?	Veredicto y motivo
`normalizarCodigo`: quita espacios / mayúsculas / combina / solo espacios	Sí (`toBe`)	Sí (regla 2)	Sí (cadena vacía)	Conservar
`normalizarCodigo`: no elimina espacios intermedios	Sí	No: la regla no lo dice; describe el código	No	Conservar como documentación, no como regla
`codigoValido`: acepta 3 códigos / minúsculas y espacios	Sí	Sí	No	Conservar
`codigoValido`: rechaza 11 formatos (3 y 5 letras, 1 y 3 dígitos, Ñ, Ç, guion...)	Sí	Sí	Sí	Conservar: es la que mata los mutantes de la expresión regular (Ejercicio 1)
`redondear2`: dos decimales, 1.005, 0.1+0.2, enteros	Sí	Sí (regla 4, mitad hacia arriba)	Sí (1.005)	Conservar
`redondear2`: funciona con negativos	Sí	No: nunca hay importes negativos	No	Descartar: comportamiento no especificado
`aplicarCupon`: rechaza 0, -1, -0.01, NaN, ±Infinity	Sí (`toEqual` con mensaje)	Sí (regla 1)	Sí (0 y -0.01)	Conservar
valida el precio antes que el cupón nulo	Sí	No: orden interno del código	No	Descartar: fija la implementación
rechaza cuando no hay cupón / formato inválido	Sí	Sí (mensajes literales)	No	Conservar
acepta un código en minúsculas con espacios	**No: solo `r.ok`**	Parcial	No	Corregir: comprobar `descuentoBs` y `precioFinal`
valida el código antes que la fecha	Sí	No: orden interno	No	Descartar
rechaza un cupón vencido ayer	Sí	Sí (regla 3)	Sí	Conservar
acepta el mismo día / vence mañana	**No: solo `.ok`**	Sí	Sí	Corregir: aserción del resultado completo
compara usando la fecha UTC	Sí	**No: contradice la regla 3**	Sí	**Descartar: fija el defecto** (ver §3)
aplica el porcentaje / limita al tope / igual al tope	Sí	Sí (regla 4)	Sí (tope exacto)	Conservar

Prueba (ronda 1)	¿Aserción específica?	¿Valor de la regla?	¿Límite?	Veredicto y motivo
redondea descuento y precio final (99,99 × 15 %)	Sí	Sí	Sí	Conservar
porcentaje 0 / tope 0 / 100 % deja 0	Sí	Sí	Sí	Conservar
el precio final nunca es negativo (150 %)	Sí	Sí (regla 5)	Sí	Conservar; el hueco (¿puede el descuento superar al precio?) se anotó como pregunta al negocio
helper `crearCupon` (`'PROMO1'.replace(...)`)	—	—	—	Corregir: código enrevesado; la propia IA lo admitió

Resultado: 14 filas conservar, 4 corregir, 4 descartar (una por fijar un defecto). En el pull request se conservó la suite de la **ronda 3**, que cubre las cinco reglas con aserciones completas, es determinista y alcanza 100 % de mutación en UTC; se le aplicó una corrección humana (el nombre "redondea hacia abajo" era engañoso: 1,234 → 1,23 es redondeo normal). Las pruebas útiles exclusivas de la ronda 1 (Ñ, Ç, guion) quedan como mejora pendiente: no matan ningún mutante adicional.

5. Las pruebas por el pipeline (Paso 7)

Commits: `a7b1cfc` chore (Stryker), `20a82bd` feat (módulo), `3536cf1` fix (zona horaria), `282466c` test (pruebas de IA). Husky y commitlint rechazaron dos commits por líneas de más de 100 caracteres en el cuerpo; se corrigieron sin `--no-verify`.



Captura 5 — PR #6, run 38901492866: todos los jobs del CI en verde; el JUnit del artefacto muestra 58 pruebas (5 + 39 + 14), 0 fallos, 0 omitidas. En ese primer run el preview falló solo en el smoke test porque el servicio de Render estaba suspendido; tras reactivarlo, el run 38903413904 quedó completamente verde (incluido el preview).

6. Manual de usuario (Pasos 8 a 11)

6.1 Inventario (verdad terreno)

El inventario completo está en `practica9/inventario.md` (recorrido con el navegador, sin leer el código, 16 capturas). Filas clave: título `Calculadora de Descuentos · COM450`; campos `Precio original (Bs)` (100) y `Descuento (%)` (10); botón `Calcular` / `Calculando…`; resultado `Precio final: Bs 150,00`; errores `El precio original debe ser mayor que cero` y `El porcentaje de descuento debe estar entre 0 y 100`; pie `API 1.0.0 · commit 532d643` / `API sin conexión`; sin API, el error `Failed to fetch`.

Corrección del propio inventario. La primera versión decía que el precio "acepta decimales (33,33)". El recorrido automático escribía `33.33`. Al verificar el manual se probó la coma: el campo

numérico la **descarta sin aviso** (33,33 → 3333 → `Bs 2999,70`). También se descubrió que el **descuento vacío se toma como 0 %** sin aviso y que `12,5` en el descuento se lee como 125. El inventario se corrigió y lo anoto como lección: la verdad terreno también se verifica.

6.2 Generación (versión IA 1) y verificación

Prompt: plantilla del Anexo B + inventario + 6 capturas. La tabla completa de 43 afirmaciones está en el Anexo 2.

Tipo	Cantidad	Ejemplos
Correctas	**32 (74,4 %)**	nombres de campos, botón, mensajes literales, formato `Bs 1111,05`, redondeo
Dato incorrecto	4	"admiten decimales, por ejemplo 33,33"; commit fijo en el pie; "revise su internet" cuando falla el servidor
Formato	4	ejemplos de entrada con coma (`33,33`, `0,05`, `1234,5`)
Omisión	3	dirección de la aplicación; precio máximo; no avisa que los decimales van con punto
Alucinación	0	—

El 74,4 % no es "casi bien": los 11 errores se concentran en lo que más le importa al usuario (cómo escribir un precio con centavos) y uno de ellos produce un resultado 100 veces mayor sin ningún aviso.

Captura 6 — Manual frente a la aplicación real (decimales)



Aplicación real (evidencia)

IA — versión IA 1 (Claude)
Los dos campos solo aceptan números y admiten decimales (por ejemplo 33,33).
FAQ 2: Sí, los campos aceptan decimales, por ejemplo 33,33.

Prueba en la aplicación
Se escribe 33,33 con 10 %: el campo descarta la coma, queda 3333 y el resultado es «Precio final: Bs 2999,70» sin ningún aviso (con 33.33 da Bs 30,00).

Versión final (manual-final)
Importante: los centavos se escriben con punto. Escriba 33.33, no 33,33. Si usa coma, el campo la borra sin avisar: toma 3333 y el resultado sale mal (Bs 2999,70 en lugar de Bs 30,00).

Captura 6 — La aplicación real frente al fragmento de la versión IA 1 y de la versión final.

Versión final: `practica9/manual-final.html` y `manual-final.pdf` (8 páginas, capturas reales, aviso de la coma, descuento vacío, preguntas frecuentes y referencia rápida).

6.3 Prueba de lectura (Paso 11)

> **Pendiente de realizar por el estudiante** con una persona ajena a la carrera, usando solo `manual-final.pdf`. Tareas: (1) calcular el precio final de 200 Bs con 25 %; (2) provocar un error y explicar qué significa.

Dato	Anotación
Persona (relación, sin nombre)	
Tiempo tarea 1 / tarea 2	
Dónde dudó	
Qué no entendió	
Qué pidió que se agregue	
Corrección aplicada al manual	

7. Bitácora de uso de IA (Paso 12)

Ver Anexo 1.

8. Reto de pruebas: la IA que miente (Parte B)

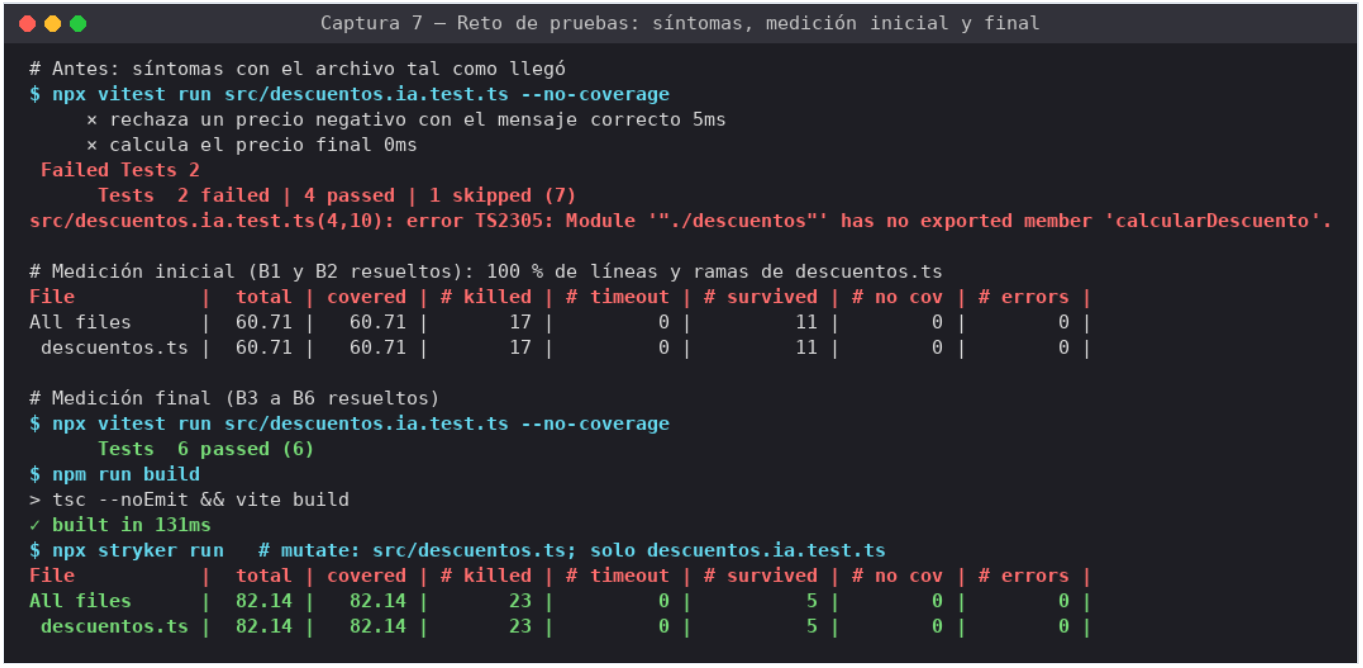
Síntomas al copiar `descuentos.ia.test.ts`: 2 fallos, 1 omitida, `tsc` con `TS2305 … no exported member 'calcularDescuento'`, sin informe de cobertura.

Defecto	Clasificación	Antes	Después	Commit
B1	Alucinación (dato inventado: mensaje en inglés)	`toBe('Original price must be greater than zero')`	`toBe('El precio original debe ser mayor que cero')`	`b38e93a`
B2	Alucinación (API inexistente)	`import { calcularDescuento … } from './descuentos'`; `expect(calcularDescuento(200, 25)).toBe(150)`	Prueba eliminada: el cálculo vive en la API Java (`CalculadoraDescuentos`, ya probada con 200/25 → 150) y `api.ts` solo tiene un cliente HTTP asíncrono. Única eliminación permitida	`00fcd92`
B3	Aserción vacía	`expect(validar(0, 10)).toBeDefined()` (pasa con `null`)	`toBe('El precio original debe ser mayor que cero')`	`bf7053e`
B4	Prueba sin aserción	`validar(100, 150)`	`expect(validar(100, 150)).toBe('El porcentaje de descuento debe estar entre 0 y 100')`	`5426528`
B5	Prueba omitida en el límite	`it.skip('rechaza el porcentaje 100.01', …)`	`it(…)`	`ce2b62e`
B6	Tautología (repite la fórmula)	`toBe(`Bs ${valor.toFixed(2).replace('.', ',')}`)`	`toBe('Bs 1234,50')`	`d37fc64`

El archivo se confirmó primero sin cambios (`ed2e000`) y cada defecto en su propio commit.

Medición (solo `descuentos.ia.test.ts` contra `descuentos.ts`)	Pruebas	Líneas	Mutación	Sobrevivientes
Inicial (B1 y B2 resueltos)	5 + 1 omitida	100 % (ramas 100 %)	**60,71 %** (17/28)	11
Final (B3 a B6 resueltos)	6, 0 omitidas, `tsc` limpio	100 %	**82,14 %** (23/28)	5

Los 5 sobrevivientes finales están todos en la condición del porcentaje (`!Number.isFinite(porcentaje) || porcentaje < 0 || porcentaje > 100`): `porcentaje <= 0`, `porcentaje >= 100`, `false || porcentaje > 100`, `… && porcentaje < 0` y `!Number.isFinite(...) || false`. Sobreviven porque el archivo de la IA nunca prueba un porcentaje negativo, ni NaN, ni los extremos 0 y 100 aceptados. No se agregaron pruebas nuevas (el reto pide corregir, no completar); la suite de la Práctica 8 sí los mata (línea base 100 %).



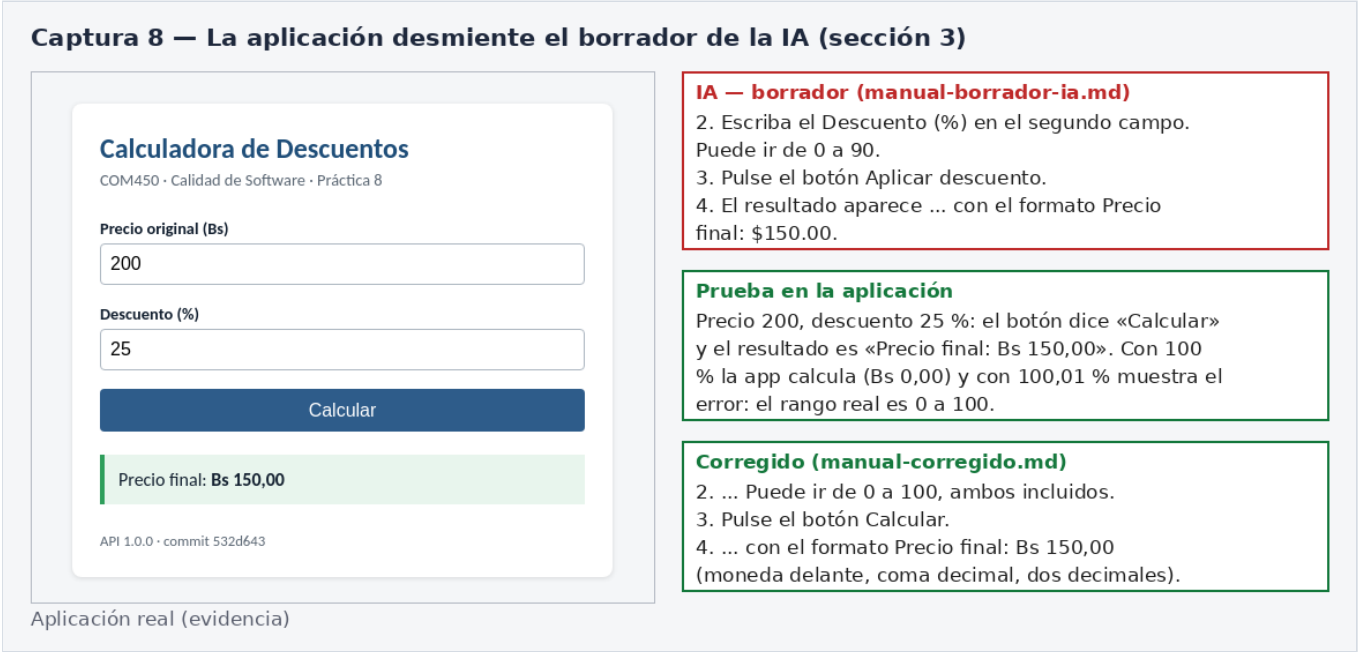
Captura 7 — Síntomas, medición inicial (60,71 %) y final (82,14 %, 6/6 en verde, `tsc` limpio). `stryker.config.json` se restauró antes de confirmar (`git restore`).

9. Reto del manual: ocho errores (Parte B)

N.º	Afirmación del borrador	Realidad (evidencia)	Tipo	Corrección	Riesgo
M1	§2: "debe iniciar sesión con su correo institucional… pulse **Crear cuenta**"	No hay inicio de sesión ni cuentas: el único botón es `Calcular` (inventario, Captura 01)	Alucinación	"No hace falta crear cuenta ni iniciar sesión"	Alto: el usuario busca un acceso que no existe y abandona
M2	§3: "El descuento puede ir de 0 a 90"	0 a 100 inclusive: 100 % da `Bs 0,00`; 100,01 da error (Capturas 06 y 12)	Dato incorrecto	"de 0 a 100, ambos incluidos"	Medio: cree que no puede aplicar 95 %
M3	§3: "Pulse el botón **Aplicar descuento**"	El botón dice `Calcular` (Captura 02)	Dato incorrecto	"Pulse el botón **Calcular**"	Medio: busca un botón inexistente
M4	§3: formato "**Precio final: $150.00**"	`Precio final: Bs 150,00` (Captura 02)	Formato	`Bs 150,00`: moneda delante, coma decimal	Medio: moneda equivocada
M5	§3: "se redondea siempre hacia abajo… 16,999 → 16,99"	Mitad hacia arriba: 0,05 con 10 % (0,045) → `Bs 0,05`; 33.33 con 10 % (29,997) → `Bs 30,00` (Capturas 03 y 04; regla de la Práctica 6)	Dato incorrecto	Redondeo de la mitad hacia arriba con los dos ejemplos	Medio: espera un centavo menos
M6	§4: mensaje único "**Ingrese valores válidos**" en rojo	Dos mensajes literales: `El precio original debe ser mayor que cero` y `El porcentaje de descuento debe estar entre 0 y 100`, en recuadro rosado (Capturas 08 y 10)	Dato incorrecto	Tabla con los dos mensajes, causa y qué hacer	Medio: no sabe qué campo corregir

N.º	Afirmación del borrador	Realidad (evidencia)	Tipo	Corrección	Riesgo
M7	§5: "guarda los últimos 10 cálculos... pulse **Historial**"	No existe historial ni ese botón (recorrido de la página)	Alucinación	Sección eliminada; nota "no guarda historial"	Alto: confía en un registro que no existe
M8	§6: el pie muestra "la fecha de la última actualización y el nombre del desarrollador"	Muestra `API 1.0.0 · commit 532d643` o `API sin conexión` (Capturas 01 y 15)	Dato incorrecto	Describir los dos estados del pie	Bajo, pero oculta el estado "sin conexión"

Omisiones anotadas aparte: el estado `Calculando…`, el error `Failed to fetch` sin API y que los decimales se escriben con punto. Corrección: `practica9/manual-corregido.md` (commits `e1568f7` y `c92bb9f`).



Captura 8 — La aplicación desmiente M2, M3 y M4.

10. Ejercicios propuestos (Parte C)

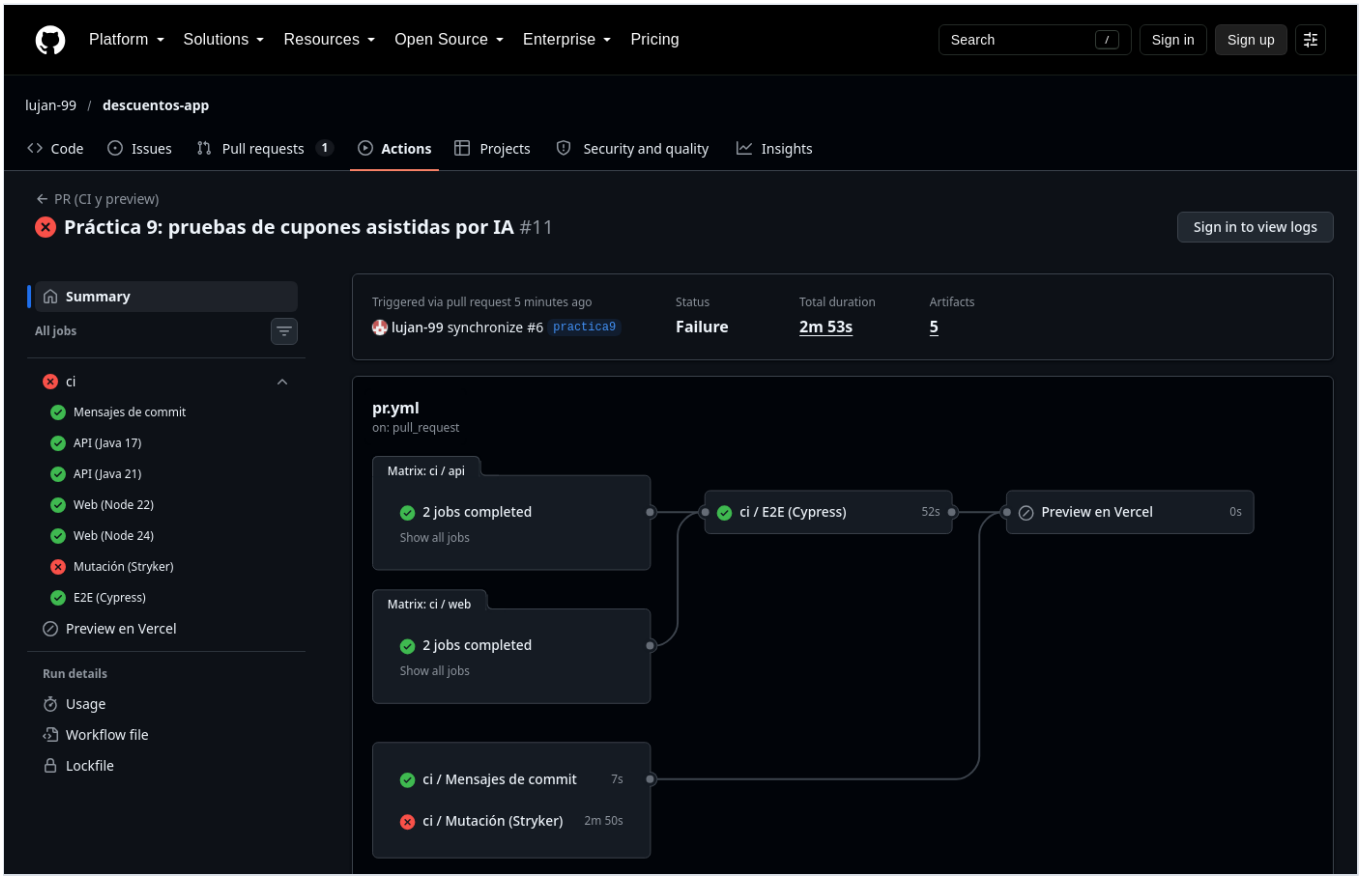
Ejercicio 1 — La mutación como control del pipeline

Job `mutacion` en `ci.yml` (commits `59be3cf` y `04d9252`): solo en pull requests, `TZ: UTC`, `npx stryker run --mutate src/cupones.ts --concurrency 2 --timeoutMS 20000`, informe como artefacto; `thresholds.break: 98` en `stryker.config.json`.

- **¿Por qué 98 y no 70?** Con 70 ninguna prueba quitada ponía el job en rojo: sin las 11 pruebas de códigos rechazados el puntaje queda en 96,61 % (la suite es redundante). Un umbral muy por debajo del puntaje real no protege nada; se usó un umbral tipo trinquete justo debajo del valor alcanzado.
- **Rojo:** commit `60876d2` quita esas pruebas → run 38950990225: **96,61 % < 98**, sobreviven los mutantes `/^[A-Z]{4}[0-9]{2}/` y `/[A-Z]{4}[0-9]{2}$/` (la expresión pierde un ancla), exit code 1.
- **Verde:** `revert ac187a1` → el job vuelve a 100 %.
- **Hallazgo:** el primer intento de rojo salió **verde** con 98,31 %: con `--concurrency 4` en un runner de 2 núcleos, 46 mutantes terminaron en *timeout*, que Stryker cuenta como detectados. El control daba falsa seguridad hasta que se bajó la concurrencia (commit `04d9252`).

Ejercicio 1 — Mutación en rojo (96,61 % < 98)

```
$ # run 38950990225 — commit 60876d2 (quita las pruebas de códigos rechazados) — job ci / Mutación (Stryker)
[Survived] Regex
src/cupones.ts:25:23
-   const PATRON_CODIGO = /^[A-Z]{4}[0-9]{2}$/
+   const PATRON_CODIGO = /^[A-Z]{4}[0-9]{2}/
[Survived] Regex
src/cupones.ts:25:23
-   const PATRON_CODIGO = /^[A-Z]{4}[0-9]{2}$/
+   const PATRON_CODIGO = /[A-Z]{4}[0-9]{2}$/
File        |  total | covered | # killed | # timeout | # survived | # no cov | # errors |
All files   |  96.61 |   96.61 |       57 |         0 |          2 |        0 |        0 |
 cupones.ts |  96.61 |   96.61 |       57 |         0 |          2 |        0 |        0 |
18:57:50 (2368) ERROR MutationTestReportHelper Final mutation score 96.61 under breaking threshold 98, setting exit code to 1 (failure).
18:57:50 (2368) INFO MutationTestExecutor Done in 2 minutes and 32 seconds.
##[error]Process completed with exit code 1.
```



Ejercicio 1 — Mutación en verde tras el revert (100 %)

```
$ # run 38956860256 — commit ac187a1 (revert) — job ci / Mutación (Stryker)
File        |  total | covered | # killed | # timeout | # survived | # no cov | # errors |
All files   | 100.00 |  100.00 |       59 |         0 |          0 |        0 |        0 |
 cupones.ts | 100.00 |  100.00 |       59 |         0 |          0 |        0 |        0 |
19:45:04 (2420) INFO MutationTestReportHelper Final mutation score of 100.00 is greater than or equal to break threshold 98
19:45:04 (2420) INFO MutationTestExecutor Done in 2 minutes and 34 seconds.
Práctica 9: pruebas de cupones asistidas por IA · lujan-99/descuentos-app@ac187a1 · GitHub
```

¿Por qué solo en pull requests o sobre archivos modificados? Cada mutante ejecuta la suite completa (ejecutor `command`): el job tarda 2 min 13 s – 2 min 32 s solo para `cupones.ts` (59 mutantes), más que el resto del CI juntos. En `main` el código ya pasó por el PR; repetirlo no agrega información. **Cómo reducir el costo:** mutar solo los archivos del diff (`--mutate $(git diff --name-only origin/main -- 'web/src/*.ts')`), modo `--incremental` con el informe anterior en caché, el ejecutor de Vitest de Stryker con `coverageAnalysis: perTest` (cada mutante corre solo las pruebas que lo cubren) y excluir del comando las pruebas lentas (`App.test.tsx`).

Ejercicio 2 — E2E de Cypress generado por IA

Prompt con el inventario y los `data-cy` (Anexo 1). Resultado (`web/cypress/e2e/ia.cy.ts`, commit `3364604`):

Pregunta	Respuesta
¿Cuántas pasan en las cinco corridas?	3/3 en las 5 corridas (y la versión corregida 4/4 en otras 5)
¿Alguna inestable?	No; ~2 s por corrida, siempre igual
¿Selectores inexistentes?	No: solo `data-cy` reales
¿`cy.wait(n)` fijos?	No; usa reintentos con `timeout: 10000`

Corrección y motivo: el `beforeEach` comprobaba el título, los valores iniciales y el texto `Calcular`. Al cambiar un texto (Ejercicio 3) fallaban las tres pruebas a la vez en el gancho y no se ejecutaba ninguna. Se movió el estado inicial a una prueba propia y el `beforeEach` quedó solo con `cy.visit('/')`. Hallazgo de entorno: `cy.screenshot()` al fallar se colgó 30 s en modo headless; se corrió con `screenshotOnRunFailure=false`.

Ejercicio 3 — Selectores frágiles frente a estables

La IA, viendo solo dos capturas, respondió con `cy.get('input').eq(0)`, `cy.get('input').eq(1)` (posición) y `cy.contains('button', 'Calcular')` (texto). Con el botón cambiado a "Aplicar":

Prueba	Selectores	Resultado
`descuento.cy.ts` (Práctica 4/8)	`data-cy`	**5/5 pasan**
`ia-sin-datacy.cy.ts` (IA sin `data-cy`)	posición + texto	**Falla**: "Expected to find content: 'Calcular' within the selector: 'button'"
`ia.cy.ts` original (IA con `data-cy`)	`data-cy` + aserción de texto	**Falla** en el `beforeEach` (`-'Aplicar'`)

¿Cuál se rompió? La que localiza por texto o posición. Matiz: la versión con `data-cy` también falló, pero porque **afirmaba** el texto: eso puede ser correcto si el texto es un requisito (el manual dice "pulse Calcular"). **Self-healing:** resuelve el mantenimiento de selectores que cambian (la herramienta busca el elemento por otros atributos y "repara" la prueba). Problema nuevo: una prueba que se arregla sola puede **dejar de verificar lo que debía** —si el botón pasa a decir otra cosa o se cambia por otro elemento, la herramienta lo acepta y la regresión no se reporta—, y la reparación queda fuera de la revisión humana si nadie la confirma.

Ejercicio 4 — Ayuda contextual y referencia rápida

Generado con Claude a partir del inventario corregido (texto completo en `practica9/ejercicios/ej4-respuesta-ia.md`).

Texto generado	Verificación en la aplicación	Veredicto
Tooltip precio: "...mayor que cero. Para decimales use punto, no coma: 33.33"	Captura `verificacion/*-coma.png`	Correcto
Tooltip descuento: "de 0 a 100. Para decimales use punto, no coma"	`12,5` → 125 → error (`verificacion/descuento-coma.png`)	Correcto
Tooltip botón: "Muestra el precio final... Mientras dice «Calculando...», espere"	Captura 14	Correcto
Error precio reescrito: "Falta el precio, o es cero o negativo..."	Capturas 08, 09, 13	Correcto

Texto generado	Verificación en la aplicación	Veredicto
Error porcentaje reescrito: "...no puede ser menor que 0 ni mayor que 100..."	Capturas 10–12	Correcto, pero incompleto: no explica que `12,5` produce este error
Referencia rápida: pasos, reglas, mensajes, pie, "no hay historial"	Inventario	Correcta
Referencia: "`Failed to fetch` → NO VERIFICADO"	Captura 16	Corrección humana: "el servidor no responde; espere y recargue"
Referencia: pie `API 1.0.0 · commit 532d643`	Cambia en cada despliegue	Corrección humana: "API seguida de su versión"
"Descuento vacío: NO VERIFICADO"	Vacío → 0 % sin aviso (`Bs 100,00`)	Hallazgo nuevo; agregado al manual final

¿Qué requirió corrección humana? Lo que el inventario no cubría (qué hacer sin servidor, descuento vacío, coma en el descuento) y los datos que cambian con cada versión. **¿Cómo mantenerlo correcto?** Tratar el manual como código: textos y mensajes en un solo archivo de cadenas que usan la app y el manual; una prueba E2E por cada afirmación crítica del manual (botón, mensajes, formato) para que un cambio de texto rompa el CI; regenerar las capturas con el script en cada versión y no copiar números de versión en el manual.

Ejercicio 5 — Política de uso de IA del equipo

Ver Anexo 3.

Ejercicio 7 — Análisis escrito

1. Cobertura alta y mutación más baja. En esta práctica la ronda 1 obtuvo 100 % en ambas, así que la pregunta se responde con el reto, donde sí ocurrió: el archivo de la IA cubría el 100 % de las líneas y ramas de `descuentos.ts` y solo detectaba el 60,71 % de los mutantes. Ejemplo concreto: el mutante `precio <= 0` → `precio < 0` sobrevivía porque la prueba "rechaza el precio cero" usaba `toBeDefined()`, que también pasa con `null` (el valor que devuelve `validar` cuando el dato es válido). La línea se ejecutaba (cobertura) pero nadie miraba el resultado (mutación). Y el caso de `cupones.ts` agrega una lección más fuerte: 100 % de mutación tampoco garantiza nada si el valor esperado sale del código; la ronda 1 tenía 100 % y fijaba el defecto de zona horaria.

2. Darle las reglas de negocio en la ronda 2. Gané: la IA dejó de describir el código y empezó a juzgarlo; encontró el defecto de UTC y propuso límites (199,90 / 200 / 200,10 para el tope). Perdí: menos pruebas de detalle (`redondear2` directo, caracteres como Ñ o Ç) y una dependencia nueva del entorno (hora local). Si la regla estuviera mal escrita, la IA la aplicaría al pie de la letra con la misma confianza: por ejemplo, la regla 4 no dice si el descuento puede superar al precio, y la IA fijó `descuentoBs: 150` sobre 100 Bs como esperado. La IA traslada el error de la especificación a la prueba; por eso la regla también se revisa (la regla 3 se tuvo que precisar con la zona horaria).

3. Tiempo de generar frente a verificar. Generar: cada respuesta de la IA tardó entre 14 s y 2 min (las tres rondas, cuatro muestras, tres manuales y los ejercicios sumaron menos de 15 minutos de generación). Verificar: medir cada ronda (Stryker 30 s – 2,5 min por corrida, repetido por zona horaria), revisar 22 filas de pruebas, recorrer la app para el inventario, probar 43 afirmaciones del manual con capturas y repetir las verificaciones cuando apareció la coma. En proporción, más del 85 % del tiempo fue verificar. La IA ahorra la escritura y la estructura; traslada el trabajo a la medición, a la revisión de valores esperados y a la investigación de lo que nadie le dijo (zona horaria, coma, descuento vacío).

4. Un manual con 85 % de afirmaciones correctas. Para el usuario no existe el promedio: existe la afirmación que está leyendo. Si el 15 % restante es "el número de commit del pie", da igual; si es "puede escribir 33,33", el usuario obtiene un precio 100 veces mayor sin ningún aviso y cobra mal. Nuestra versión IA 1 tuvo 74,4 % y sus errores estaban justo en el paso más usado. Un manual se mide por el riesgo de sus errores, no por el porcentaje: 100 % de las afirmaciones de alto riesgo verificadas es la condición mínima.

5. Cuándo confiar. Confiar en lo generado solo cuando pasó por los mismos controles que el código humano y nadie los debilitó para que pase: es la misma regla de la Práctica 3 (no se apagan reglas de SonarQube para aprobar el Quality Gate), de la Práctica 4 (no se modifica un spec de Cypress para que pase, se corrige la app o se justifica el cambio de requisito) y de la Práctica 8 (no se usa `--no-verify`). Concretamente: (a) confiar en la estructura y en la sintaxis cuando compila y pasa; (b) no confiar en ningún valor esperado que no tenga una regla o una observación de la app como fuente; (c) no confiar en un puntaje sin mirar sus sobrevivientes ni sus timeouts (Ejercicio 1); (d) no confiar en un manual sin verificar cada afirmación de alto riesgo. En esta práctica el único "atajo" tentador fue bajar el umbral de mutación a 70: habría dejado el job siempre en verde.

11. Enlaces y entregables

- Repositorio: https://github.com/lujan-99/descuentos-app/tree/practica9 — Pull request: https://github.com/lujan-99/descuentos-app/pull/6
- `practica9/manual-final.html` y `manual-final.pdf`; `practica9/manual-corregido.md`; `practica9/inventario.md`
- Presentación: `practica9/presentacion.pdf`

Anexo 1 — Bitácora de uso de IA

Fecha y hora	Herramienta y modelo	Para qué	Prompt (resumen; texto exacto en `practica9/rondas` y `ejercicios`)	Qué se corrigió a mano
16/10 08:07	Claude Opus 5.5 (Claude Code, conversación nueva)	Ronda 1	"Genera pruebas unitarias con Vitest para este archivo TypeScript:" + código sin cabecera	Ruta del import `./cupon` → `./cupones`. Revisión: 4 descartar, 4 corregir
16/10 08:10	Claude Opus 5.5	Ronda 2	Plantilla del Paso 6 con las reglas + código	Nada para compilar; se midió en UTC porque fallaba en La Paz
16/10 08:15–08:21	Claude Opus 5.5; OpenCode 2.0.29 con DeepSeek V4.1 Flash, MiMo v2.6 Flash, Big Pickle (Nemotron 3 Ultra y Step 5: error 400)	Muestra de la matriz A	"Genera pruebas unitarias con Vitest para este archivo TypeScript (descuentos.ts). Devuelve solo el archivo de pruebas." + código	Nada (se midió tal cual). OpenCode en modo *build* ejecutó `npm install vitest` sin permiso: desde entonces se usó `--agent plan`
16/10 08:24	Claude Opus 5.5	Ronda 3	Reglas + código + pruebas de la ronda 2 + evidencia (0 sobrevivientes; fallo en TZ La Paz)	Renombre de "redondea hacia abajo"; corrección de `cupones.ts` decidida por el estudiante
16/10 08:41	Claude Opus 5.5; OpenCode DeepSeek y MiMo (`--agent plan`)	Manual (versión IA 1) y muestra de la matriz B	Plantilla del Anexo B + inventario + 6 capturas	11 errores corregidos (Anexo 2); se corrigió también el inventario
16/10 09:05	Claude Opus 5.5	Ejercicio 2	Tres E2E con inventario y lista de `data-cy`	`beforeEach` con aserciones → prueba propia
16/10 09:05	Claude Opus 5.5	Ejercicio 3	E2E viendo solo dos capturas	Ninguna (se dejó frágil a propósito)
16/10 09:06	Claude Opus 5.5	Ejercicio 4	Referencia rápida + 3 tooltips + 2 mensajes con el inventario	`Failed to fetch`, versión del pie, descuento vacío
16/10 (toda la sesión)	Claude Code (Opus 5.5)	Asistente de toda la práctica: comandos, capturas, scripts de medición, borrador del informe y diapositivas	Instrucción del estudiante: realizar la práctica de punta a punta	Revisión del estudiante; declarado en la portada

No se pegó en ningún prompt el token de Vercel, los deploy hooks ni datos personales: los prompts solo contienen el código del módulo (material del curso), las reglas y capturas de la aplicación con datos de ejemplo.

Anexo 2 — Tabla de verificación del manual (versión IA 1)

N.º	Sección	Afirmación del manual	Realidad (evidencia)	Tipo	Corrección	Riesgo para el usuario
1	Cabecera	Versión descrita: `API 1.0.0 · commit 532d643`	Cierto en local; en producción el commit cambia con cada despliegue (inventario, pie)	Dato incorrecto	Decir "API <versión> · commit <código>" sin fijar el número	Bajo: el usuario cree que tiene otra versión

N.º	Sección	Afirmación del manual	Realidad (evidencia)	Tipo	Corrección	Riesgo para el usuario
2	1	Escribe el precio en Bs y el porcentaje; muestra el precio final	Captura 02	Correcta	—	—
3	1	Bajo el título dice `COM450 · Calidad de Software · Práctica 8`	Captura 01	Correcta	—	—
4	1	No hay inicio de sesión, cuenta, historial, menú ni enlaces; el único botón es `Calcular`	Recorrido: único botón "Calcular"	Correcta	—	—
5	2	Título de la pestaña `Calculadora de Descuentos · COM450`	document.title	Correcta	—	—
6	2	Encabezado `Calculadora de Descuentos`	Captura 01	Correcta	—	—
7	2	Campo `Precio original (Bs)` con valor inicial `100`	Captura 01	Correcta	—	—
8	2	Campo `Descuento (%)` con valor inicial `10`	Captura 01	Correcta	—	—
9	2	Botón `Calcular` hace el cálculo	Captura 02	Correcta	—	—
10	2	Pie: con conexión muestra la versión de la API; sin conexión `API sin conexión`	Capturas 01 y 15	Correcta	—	—
11	2	Los dos campos solo aceptan números	"abc" no se escribe (verificacion/*-letras.png)	Correcta	—	—
12	2	Admiten decimales "por ejemplo `33,33`"	Con coma el campo queda en 3333 y calcula Bs 2999,70 sin aviso (verificacion/*-coma.png)	Dato incorrecto	Decimales con punto: `33.33`; advertir que la coma se descarta	Alto: resultado 100 veces mayor sin ningún mensaje
13	3.1	Dirección web de la página: NO VERIFICADO	La aplicación está publicada en https://descuentos-web-blush.vercel.app	Omisión	Incluir la dirección	Medio: el usuario no sabe dónde entrar
14	3.2	Si el pie muestra la versión, está lista; si dice `API sin conexión`, ir a la sección 5	Capturas 01 y 15	Correcta	—	—
15	3.3	Borrar el valor y escribir un precio mayor que 0	Captura 08 (0 da error)	Correcta	—	—
16	3.4	Descuento entre 0 y 100	Capturas 05, 06 y 12	Correcta	—	—

N.º	Sección	Afirmación del manual	Realidad (evidencia)	Tipo	Corrección	Riesgo para el usuario
17	3.6	Mientras calcula el botón dice `Calculando…`, se pone gris y no se puede pulsar	Captura 14 (gris azulado, deshabilitado)	Correcta	—	—
18	3.7	El resultado sale en un recuadro verde debajo del botón	Captura 02	Correcta	—	—
19	3.7	200 y 25 dan `Precio final: Bs 150,00`	Captura 02	Correcta	—	—
20	3	Siempre empieza con Bs; coma decimal y siempre 2 decimales	Capturas 02, 05, 06	Correcta	—	—
21	3	Sin separador de miles: `Bs 1111,05`	Captura 07	Correcta	—	—
22	3	Redondeo a dos decimales, la mitad hacia arriba	Capturas 03 y 04	Correcta	—	—
23	3	Ejemplo "`33,33` con 10 % da `Bs 30,00`"	El cálculo es cierto con 33.33; si el usuario copia el ejemplo con coma obtiene Bs 2999,70	Formato	Escribir la entrada como `33.33`	Alto: el ejemplo enseña a escribir mal
24	3	Ejemplo "`0,05` con 10 % da `Bs 0,05`"	Cierto con 0.05; con coma el campo toma 5 y da Bs 4,50	Formato	Escribir la entrada como `0.05`	Alto
25	4	Los errores salen en un recuadro rosado con borde rojo debajo de `Calcular`	Captura 08	Correcta	—	—
26	4	Mensaje literal `El precio original debe ser mayor que cero`	Captura 08	Correcta	—	—
27	4	Aparece con precio 0, negativo (-5) o vacío	Capturas 08, 09, 13	Correcta	—	—
28	4	Qué hacer: escribir un precio mayor que 0 "por ejemplo `0,05`"	Con coma se acepta como 5: no corrige el error, lo esconde	Formato	Ejemplo `0.05`	Medio
29	4	Mensaje literal `El porcentaje de descuento debe estar entre 0 y 100`	Captura 10	Correcta	—	—
30	4	Aparece con más de 100 (150, 100,01) o negativo (-1)	Capturas 10, 11, 12	Correcta	—	—
31	4	`Failed to fetch` está en inglés y significa que no hubo conexión con el servicio	Captura 16	Correcta	—	—
32	5	Con `API sin conexión` la calculadora no puede calcular	Capturas 15 y 16	Correcta	—	—

N.º	Sección	Afirmación del manual	Realidad (evidencia)	Tipo	Corrección	Riesgo para el usuario
33	5	Qué hacer: "revise que su computadora o celular tenga internet"	Si la página cargó, hay internet: la causa es el servidor (p. ej. Render suspendido o dormido)	Dato incorrecto	Decir que el servidor no responde y que espere unos minutos y recargue	Bajo: busca un problema que no tiene
34	5	Recargar la página y mirar si vuelve la versión en el pie	Al recargar se vuelve a consultar la API (Captura 01 tras reactivarla)	Correcta	—	—
35	FAQ 1	Con 100: 0 % da `Bs 100,00` y 100 % da `Bs 0,00`	Capturas 05 y 06	Correcta	—	—
36	FAQ 2	"Los campos aceptan decimales, por ejemplo `33,33`"; "`0,05` se acepta"	La coma se descarta (verificacion/*-coma.png)	Dato incorrecto	Explicar que se usa punto	Alto
37	FAQ 3	"`1234,5` con 10 % se muestra como `Bs 1111,05`"	Cierto con 1234.5; con coma da Bs 11110,50	Formato	Entrada `1234.5`	Medio
38	FAQ 4	No se guardan cálculos ni hace falta cuenta	Recorrido	Correcta	—	—
39	FAQ 5	`Failed to fetch` no es error del usuario; el pie dirá `API sin conexión`	Capturas 15 y 16	Correcta	—	—
40	Ref.	Precio máximo: NO VERIFICADO	No hay máximo: 999999999999 se calcula (verificacion/*-grande.png)	Omisión	"Sin máximo"	Bajo
41	Ref.	Descuento mínimo 0 y máximo 100, incluidos; valores iniciales 100 y 10	Capturas 01, 05, 06	Correcta	—	—
42	Ref.	Formato del resultado: Bs delante, coma decimal, 2 decimales, sin miles, mitad hacia arriba	Capturas 02, 03, 04, 07	Correcta	—	—
43	Todo	No dice que los decimales se escriben con punto aunque el resultado se lee con coma	Inventario (fila "Decimal con coma")	Omisión	Agregar una advertencia en los pasos y en la referencia rápida	Alto

Afirmaciones verificadas: 43. Correctas: 32 (74.4 %). Errores por tipo: Dato incorrecto: 4, Formato: 4, Omisión: 3.

Anexo 3 — Política de uso de IA (Ejercicio 5)

Alcance. Todo uso de asistentes de IA (chat, IDE, agentes de terminal, herramientas de documentación) en el proyecto descuentos-app.

1. Qué se puede pegar en un prompt. Sí: código del repositorio público, reglas de negocio, capturas con datos de ejemplo, mensajes de error. **No:** el token de Vercel, `VERCEL_ORG_ID` / `PROJECT_ID`, los deploy hooks de Render, claves de API, archivos `.env`, datos personales de usuarios reales, código o documentos de un cliente sin autorización escrita, ni nada que no se publicaría en un repositorio abierto. Los agentes que ejecutan comandos se usan en modo de solo lectura (`plan`) o en una carpeta aislada, nunca con acceso a credenciales.

2. Herramientas y privacidad. Solo herramientas evaluadas en la matriz; entrenamiento desactivado cuando el servicio lo permite. Modelos "gratuitos" cuyos datos se usan para entrenar: solo con material público.

3. Revisión obligatoria. Nada generado entra a `main` sin: (a) la lista de control del Anexo B (pruebas o manual), hecha por una persona distinta de quien hizo el prompt cuando el equipo lo permite; (b) el pipeline completo. Los valores esperados deben citar la regla o la observación de la app de la que salen.

4. Controles automáticos obligatorios. Cobertura ≥ 80 % de líneas exigida por Vitest; mutación con `break` igual al puntaje alcanzado menos 2 puntos en los módulos con reglas de negocio (hoy `cupones.ts`, 98); 0 pruebas omitidas; `tsc` limpio; Conventional Commits. Prohibido bajar umbrales o usar `--no-verify` para que pase código generado.

5. Registro. Bitácora por interacción relevante (fecha, herramienta y modelo, propósito, prompt o enlace, qué se corrigió) y la línea "Asistido por IA (herramienta)" en el cuerpo del commit.

6. Licencia y autoría. La responsabilidad de lo entregado es del equipo. En proyectos para clientes, antes de usar la herramienta se confirma en el contrato si se admite código generado; se usan los filtros de coincidencia con código público cuando existen.

Relación con el Plan de Calidad (Práctica 7). La política va en la sección de **Herramientas, técnicas y metodologías** del SQAP (con una referencia desde **Revisiones y auditorías** para el punto 3 y desde **Gestión de riesgos** para la fuga de información). **Métricas de cumplimiento:** (1) % de commits con "Asistido por IA" que tienen fila en la bitácora (meta 100 %); (2) % de PR con código generado que pasaron la lista de control (meta 100 %); (3) puntaje de mutación de los módulos con reglas ≥ umbral en cada PR; (4) incidentes de secretos en prompts o en el repositorio (meta 0, revisado con el escaneo de secretos de GitHub).